Proof of Theorem 17.12

- Thus if $\boldsymbol{Z} = \boldsymbol{Y} - \boldsymbol{X} \geq \boldsymbol{0}$, then $(\boldsymbol{I} - \gamma \boldsymbol{P}_{\pi_{n+1}})\boldsymbol{Z} = \sum_{k=0}^{k=\infty} (\gamma \boldsymbol{P}_{\pi_{n+1}})^k \boldsymbol{Z} \geq 0$ since the entries of $\boldsymbol{P}_{\pi_{n+1}}$ and its powers are all non-negative as well as those of $\boldsymbol{Z}$.
- By the definition of π_{n+1} we can see that $\boldsymbol{R}_{\pi_{n+1}} + \gamma \boldsymbol{P}_{\pi_{n+1}} \boldsymbol{V}_n \geq \boldsymbol{R}_{\pi_n} + \gamma \boldsymbol{P}_{\pi_n} \boldsymbol{V}_n = \boldsymbol{V}_n$.
- This shows $\boldsymbol{R}_{\pi_{n+1}} \geq (\boldsymbol{I} - \gamma \boldsymbol{P}_{\pi_{n+1}})\boldsymbol{V}_n$.
- Since $(\boldsymbol{I} - \gamma \boldsymbol{P}_{\pi_{n+1}})^{-1}$ preserves ordering, this implies that $\boldsymbol{V}_{n+1} = (\boldsymbol{I} - \gamma \boldsymbol{P}_{\pi_{n+1}})^{-1} \boldsymbol{R}_{\pi_{n+1}} \geq \boldsymbol{V}_n$ which concludes the proof.

Analysis of policy iteration algorithm

- The algorithms runs for only a finite number of iterations since the total number of possible policies is $|A|^{|}S|$.
- Two consecutive policy values can only be equal in the last iteration of the algorithm.
- For our simple example MDP, we can set up linear equations of the form below assuming $\pi_0(1) = b, \pi_0(2) = c$.

$$V_{\pi_0}(1) = 1 + \gamma V_{\pi_0}(2)$$
$$V_{\pi_0}(2) = 2 + \gamma V_{\pi_0}(2)$$

Theorem 17.13

Let $(\boldsymbol{U}_n), n \in N$ be the sequence of policy values generated by the value iteration algorithm, and $(\boldsymbol{V}_n), n \in N)$ be the one generated by the policy iteration algorithm. If $\boldsymbol{U}_0 = \boldsymbol{V}_0$, then

$$\boldsymbol{U}_n \leq \boldsymbol{V}_n \leq \boldsymbol{V}^*$$

Proof of Theorem 17.13

- First we observe that the function ϕ is monotonic.
- To see this, let $\boldsymbol{U}$ and $\boldsymbol{V}$ be such that $\boldsymbol{U} \leq \boldsymbol{V}$, and let π be the policy such that $\phi(U) = \max_{\pi} \boldsymbol{R}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{U} = \boldsymbol{R}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{U}$.
- Then $\phi(U) \leq \boldsymbol{R}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{V} \leq \max_{\pi'} \boldsymbol{R}_{\pi'} + \gamma \boldsymbol{P}_{\pi'} \boldsymbol{V} = \phi(\boldsymbol{V})$.
- The proof is by induction on n.
- Assume that $\boldsymbol{U}_n \leq \boldsymbol{V}_n$. Then, by the monotonicity of ϕ, we have $\boldsymbol{U}_{n+1} = \phi(\boldsymbol{U}_n) \leq \phi(\boldsymbol{V}_n) = \max_{\pi} \boldsymbol{R}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{V}_n$.
- Let us now examine how the policy iteration algorithm generates $\boldsymbol{V}_{n+1}$ from $\boldsymbol{V}_n$. We have already established that $\boldsymbol{U}_{n+1} = \phi(\boldsymbol{U}_n) \leq \phi(\boldsymbol{V}_n)$ and we will now proceed to establish that $\phi(\boldsymbol{V})_{\boldsymbol{n}} \leq \boldsymbol{V}_{n+1}$ to complete the argument.

Proof of Theorem 17.13

- In the $(n+1)th$ iteration, the policy iteration algorithm finds π_{n+1} to be the maximizing policy, i.e $\pi_{n+1} = \underset{\pi}{\text{argmax}}\ \boldsymbol{R}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{V}_n$.
- The policy iteration algorithm then obtains $\boldsymbol{V}_{n+1}$ from $(\boldsymbol{I} - \gamma \boldsymbol{P}_{\pi_{n+1}})\boldsymbol{V} = \boldsymbol{R}_{\pi_{n+1}}$,
- Thus we have $\boldsymbol{V}_{n+1} = \boldsymbol{R}_{\pi_{n+1}} + \gamma \boldsymbol{P}_{\pi_{n+1}} \boldsymbol{V}_{n+1}$.
- Now $\phi(\boldsymbol{V}_n) = \boldsymbol{R}_{\pi_{n+1}} + \gamma \boldsymbol{P}_{\pi_{n+1}} \boldsymbol{V}_n \leq \boldsymbol{R}_{\pi_{n+1}} + \gamma \boldsymbol{P}_{\pi_{n+1}} \boldsymbol{V}_{n+1} = \boldsymbol{V}_{n+1}$ where the inequality from the statement of Theorem 17.12 which shows that $\boldsymbol{V}_n \leq \boldsymbol{V}_{n+1}$.

Motivation

- Machine learning often uses sensitive data (medical records, finances).
- Need strong, provable privacy guarantees for released outputs/models.
- Differential privacy (DP) gives quantifiable worst-case guarantees.

What do we want from privacy?

- Output of algorithm should not depend too strongly on any single individual's data.
- Attacker shouldn't be able to infer whether a particular record was in the dataset.
- Robustness under post-processing and composition.

Definition: ε-Differential Privacy

Definition

A randomized algorithm $\mathcal{A}$ with domain datasets satisfies ε-*differential privacy* if for all neighboring datasets D, D' that differ in one record, and for all measurable sets S of outputs,

$$\Pr[\mathcal{A}(D) \in S] \leq e^{\varepsilon} \Pr[\mathcal{A}(D') \in S]$$

Definition: ε-Differential Privacy

Definition

A randomized algorithm $\mathcal{A}$ with domain datasets satisfies *ε-differential privacy* if for all neighboring datasets D, D' that differ in one record, and for all measurable sets S of outputs,

$$\Pr[\mathcal{A}(D) \in S] \leq e^{\varepsilon} \Pr[\mathcal{A}(D') \in S]$$

- Often also consider (ε, δ)-DP for approximate privacy.
- Intuition: small ε means stronger privacy.

Neighboring Datasets

Two common definitions:

- **Add/remove** one record (unbounded DP).
- **Replace** one record (bounded DP).

We'll mostly use replace-neighbors.

Sensitivity

Definition

The (global) sensitivity of a function $f : \mathcal{D} \to \mathbb{R}^k$ is

$$\Delta f = \max_{D,D' \text{ neighbors}} \|f(D) - f(D')\|_1.$$

- Sensitivity controls how much noise we must add.

Laplace Mechanism

Given scalar-valued f with sensitivity Δf, the Laplace mechanism releases

$$\mathcal{A}(D) = f(D) + \mathrm{Lap}\left(\frac{\Delta f}{\varepsilon}\right),$$

where $\mathrm{Lap}(b)$ is Laplace noise with scale b.

Laplace Mechanism

Given scalar-valued f with sensitivity Δf, the Laplace mechanism releases

$$\mathcal{A}(D) = f(D) + \mathrm{Lap}\left(\frac{\Delta f}{\varepsilon}\right),$$

where $\mathrm{Lap}(b)$ is Laplace noise with scale b.

Theorem

Laplace mechanism satisfies ε-DP.

Laplace Mechanism

Given scalar-valued f with sensitivity Δf, the Laplace mechanism releases

$$\mathcal{A}(D) = f(D) + \mathrm{Lap}\left(\frac{\Delta f}{\varepsilon}\right),$$

where $\mathrm{Lap}(b)$ is Laplace noise with scale b.

Theorem

Laplace mechanism satisfies ε-DP.

Sketch of proof: Ratio of densities of Laplace shifts by at most e^{ε} because shift between $f(D)$ and $f(D')$ is at most Δf.

Worked Example: Private Count

Suppose $f(D)$ is count of people with property (sensitivity $= 1$).
Release count $+$ Laplace$(1/\varepsilon)$. If true count $= 42$ and $\varepsilon = 0.5$,
noise scale $= 2$. So typical noise magnitude 2.

Gaussian Mechanism ((ε, δ)-DP)

For real-valued outputs, add Gaussian noise: $\mathcal{N}(0, \sigma^2 I)$. Choose σ proportional to $\Delta f/\varepsilon$ and depends on δ.

Gaussian Mechanism ((ε, δ)-DP)

For real-valued outputs, add Gaussian noise: $\mathcal{N}(0, \sigma^2 I)$. Choose σ proportional to $\Delta f / \varepsilon$ and depends on δ.

- Useful when composing many queries (concentrated/advanced DP variants).

Composition: Sequential Composition

If $\mathcal{A}_1$ is ε_1-DP and $\mathcal{A}_2$ is ε_2-DP (on same data), then $(\mathcal{A}_1, \mathcal{A}_2)$ is $(\varepsilon_1 + \varepsilon_2)$-DP.

Composition: Sequential Composition

If $\mathcal{A}_1$ is ε_1-DP and $\mathcal{A}_2$ is ε_2-DP (on same data), then $(\mathcal{A}_1, \mathcal{A}_2)$ is $(\varepsilon_1 + \varepsilon_2)$-DP. **Proof idea:** Multiply two probability ratios, exponents add.

Advanced Composition (Informal)

Repeated composition degrades privacy sublinearly in some regimes. There is an $O(\sqrt{k})$ improvement using (ε, δ)-DP bounds:

$$\varepsilon_{total} = O\left(\varepsilon\sqrt{k\log(1/\delta)}\right).$$

This is useful for iterative algorithms (SGD, repeated queries).

Post-processing and Privacy

If $\mathcal{A}$ is ε-DP and g is any (deterministic or randomized) map, then $g \circ \mathcal{A}$ is ε-DP.

Post-processing and Privacy

If $\mathcal{A}$ is ε-DP and g is any (deterministic or randomized) map, then $g \circ \mathcal{A}$ is ε-DP. **Implication:** Noise addition followed by arbitrary computation cannot weaken privacy further.

Private Mean Estimation: Setup

Dataset $D = \{x_i\}_{i=1}^n$ with $x_i \in [0, 1]$. Want private estimate of mean $\mu = \frac{1}{n} \sum_i x_i$.

Private Mean Estimation: Setup

Dataset $D = \{x_i\}_{i=1}^n$ with $x_i \in [0, 1]$. Want private estimate of mean $\mu = \frac{1}{n}\sum_i x_i$. Sensitivity of sample mean (replace-one) is $1/n$. Add Laplace noise: $\hat{\mu} = \mu + \mathrm{Lap}(1/(n\varepsilon))$.

Private Mean: Utility

Mean squared error from noise: Var(Laplace(scale=b)) $= 2b^2$. Here $b = 1/(n\varepsilon)$ so MSE noise term $= 2/(n^2\varepsilon^2)$. Compare to sampling error $\mathrm{Var}(\hat{\mu}) = O(1/n)$.

Private Mean: Utility

Mean squared error from noise: Var(Laplace(scale=b)) $= 2b^2$. Here $b = 1/(n\varepsilon)$ so MSE noise term $= 2/(n^2\varepsilon^2)$. Compare to sampling error $\text{Var}(\hat{\mu}) = O(1/n)$. When $n \gg 1/\varepsilon^2$, privacy noise negligible.

Private Histogram

For a histogram over k bins, each bin count has sensitivity 1. Add Laplace($1/\varepsilon$) to each count (or use sparse vector techniques for many queries).

Private Histogram

For a histogram over k bins, each bin count has sensitivity 1. Add Laplace($1/\varepsilon$) to each count (or use sparse vector techniques for many queries). Utility depends on k and privacy budget; can use advanced composition or partition budget.

Local vs Central Differential Privacy

Central DP: trusted curator holds raw data, releases noisy aggregates. Local DP: each user privatizes their own data before sending it.

Local vs Central Differential Privacy

Central DP: trusted curator holds raw data, releases noisy aggregates. Local DP: each user privatizes their own data before sending it. Local DP needs more noise (higher sample complexity) but removes trusted curator.

DP and Learning: Informal View

Two principal questions:

- **How to design private learners?** (algorithms)
- **How much data is required?** (sample complexity)

Private PAC Learning: Statement

Standard PAC learning requires $n = O\left(\frac{\mathrm{VC}(\mathcal{H})+\log(1/\delta)}{\alpha^2}\right)$. With DP constraints, extra factors in $1/\alpha$, $1/\varepsilon$ may appear depending on the problem.

Private Learnability: A Key Result

Theorem (Kasiviswanathan et al., 2008)

Every (non-interactive) PAC-learnable class $\mathcal{H}$ is privately learnable under (ε, δ)-DP, but sample complexity may suffer.

Private Learnability: A Key Result

Theorem (Kasiviswanathan et al., 2008)

Every (non-interactive) PAC-learnable class $\mathcal{H}$ is privately learnable under (ε, δ)-DP, but sample complexity may suffer.

We won't derive the full theorem; we'll show intuition: use private selection (exponential mechanism) to pick hypotheses.

Exponential Mechanism

For selecting from a discrete set R with quality score $q(D, r)$ of sensitivity Δq, the exponential mechanism samples r with probability proportional to

$$\exp\left(\frac{\varepsilon q(D, r)}{2\Delta q}\right).$$

It provides utility guarantees and satisfies ε-DP.

Exponential Mechanism

For selecting from a discrete set R with quality score $q(D, r)$ of sensitivity Δq, the exponential mechanism samples r with probability proportional to

$$\exp\left(\frac{\varepsilon q(D, r)}{2\Delta q}\right).$$

It provides utility guarantees and satisfies ε-DP. Useful for private hypothesis selection/ERM.

Private Empirical Risk Minimization (ERM)

Approaches:

- Output perturbation: add noise to optimal parameters.
- Objective perturbation: add random linear term to objective and optimize.
- Exponential mechanism over discrete hypothesis class.

Private Empirical Risk Minimization (ERM)

Approaches:

- Output perturbation: add noise to optimal parameters.
- Objective perturbation: add random linear term to objective and optimize.
- Exponential mechanism over discrete hypothesis class.

Each has different utility/privacy trade-offs and assumptions.

Worked Example: Private Linear Regression (1D)

Model: minimize empirical squared loss on $D = \{(x_i, y_i)\}$.
Bounded domain $|x| \leq 1, |y| \leq 1$.

Worked Example: Private Linear Regression (1D)

Model: minimize empirical squared loss on $D = \{(x_i, y_i)\}$. Bounded domain $|x| \leq 1, |y| \leq 1$. One can use objective perturbation: add $\frac{b^T w}{n}$ where b drawn from appropriate density to achieve DP. Resulting estimator is close to non-private when n large.